

WhatsApp OTP Setup Guide

A production-focused, step-by-step guide for connecting the existing GAC Holidays React application to Meta WhatsApp Cloud API, Upstash Redis and Vercel serverless functions.

Meta Cloud API | Six-digit OTP | Five-minute expiry | Secure sessions

Version 1.0 | 11 August 2026

How to use this guide

This guide intentionally isolates WhatsApp OTP login from the rest of the approved backend roadmap. Complete the steps in order. Do not connect the production business number until the Meta test number, Redis expiry and verification flow all work end to end.

Current limitation

Until customer registration is persisted in PostgreSQL, OTP proves possession of a phone number but cannot prove that the number belongs to an existing GAC customer. The temporary implementation authenticates the number and opens the current prototype dashboard.

Target architecture

```
React customer login
  |
  v
POST /api/auth/otp/request
  |-- normalize phone to E.164
  |-- rate-limit IP and phone
  |-- generate six-digit OTP
  |-- store HMAC hash in Redis (TTL 300 seconds)
  |-- send approved Meta authentication template
  |
  v
POST /api/auth/otp/verify
  |-- verify hash and attempt count
  |-- delete OTP after success
  |-- issue HttpOnly customer session cookie
```

Contents

- Meta business and developer application setup
- Official WhatsApp number and authentication template
- Permanent Meta access token
- Upstash Redis, rate limits and environment variables
- OTP request, Meta delivery and verification workflows
- Secure customer sessions and React integration
- Vercel deployment, rollout and acceptance checks
- Troubleshooting and official references

STEP 1

Create or select the Meta business portfolio

Open Meta Business and create or select the business portfolio owned by GAC Holidays.

- 1 Open business.facebook.com and sign in with the business administrator account.
- 2 Create or select the GAC Holidays business portfolio.
- 3 Open Business Settings and complete the legal name, address, website and business email.
- 4 Open Security Center and complete business verification if Meta requests it.
- 5 Use a business-controlled email address and domain wherever possible.

Ownership rule

The Meta app, WhatsApp Business Account and official phone number should be owned by the client's business portfolio, not by an individual developer account.

STEP 2

Create the Meta developer application

- 1 Open [Meta for Developers](#).
- 2 Select My Apps, then Create App.
- 3 Choose the Business app type.
- 4 Name the app GAC Holidays Customer Authentication.
- 5 Connect the app to the GAC Holidays business portfolio.
- 6 Add the WhatsApp product and open WhatsApp -> API Setup.

Record the temporary test credentials displayed by Meta: test number, temporary access token, Phone Number ID and WhatsApp Business Account ID. Use these only to validate the API with an approved test recipient.

Do not start with the official number

First send a successful template message using Meta's test number. This prevents accidental interruption of the company's active WhatsApp service.

STEP 3

Connect the official GAC Holidays number

- 1 From WhatsApp -> API Setup, select Add phone number.
- 2 Enter the approved GAC Holidays display name and business category.
- 3 Enter the official phone number and verify it by SMS or voice.
- 4 Wait for display-name approval if required.
- 5 Add a payment method in WhatsApp Manager before production traffic.

Existing WhatsApp app warning

If the official number is already used in the WhatsApp or WhatsApp Business mobile app, review Meta's current migration or coexistence options before changing anything. Do not deregister an active business number without a rollback plan.

STEP 4

Create the authentication template

Open WhatsApp Manager -> Account tools -> Message templates -> Create template.

```
Category: Authentication
Name: gac_login_otp
Language: English
OTP delivery: Copy code
Security recommendation: Enabled
Expiration warning: 5 minutes
```

Meta controls most authentication-template wording. The approved message will normally state that the supplied value is a verification code, advise the user not to share it and show a Copy Code button.

- Submit the template for approval.
- Wait until its status is Approved.
- Record the exact template name and language code.
- Use the payload sample generated for that specific template as the final authority.

STEP 5

Create the permanent Meta system-user token

The temporary token on the API Setup page expires and must not be used for production.

- 1 Open Meta Business Settings -> Users -> System users.
- 2 Create a system user with an administrator role.
- 3 Assign the Meta app, WhatsApp Business Account and phone-number assets.
- 4 Generate an access token for the application.
- 5 Grant whatsapp_business_messaging and whatsapp_business_management.
- 6 Copy the token once and store it only in Vercel's encrypted environment settings.

Secret handling

Never commit the access token, Redis token, OTP secret or session secret to Git. Never prefix them with REACT_APP_. CRA exposes every REACT_APP_ value in the browser bundle.

STEP 6

Create the Upstash Redis database

- 1 Create an account at console.upstash.com.
- 2 Create a Redis database named gac-holidays-otp.
- 3 Select a primary region close to the Vercel functions and Indian users.
- 4 Copy the REST URL and REST token from the database details page.

Upstash's REST-based Redis client is connectionless and is designed for serverless functions such as Vercel. It is appropriate for short-lived OTP and rate-limit state.

STEP 7

Install the minimal server dependencies

```
npm install @upstash/redis @upstash/ratelimit libphonenumber-js
npm install --save-dev @vercel/node typescript
```

Minimal project structure

```
gac/
|-- api/
|   |-- auth/
|   |   |-- otp/
|   |   |   |-- request.ts
|   |   |   |-- verify.ts
|-- server/
|   |-- config.ts
|   |-- phone.ts
|   |-- redis.ts
|   |-- otp.ts
|   |-- whatsapp.ts
|   |-- session.ts
|-- src/App.js
|-- .env
|-- .env.example
|-- package.json
```

Files under `api/` become Vercel serverless endpoints. Files under `server/` contain shared server-only code.

STEP 8

Configure server-only environment variables

```
WHATSAPP_GRAPH_VERSION=v20.0
WHATSAPP_CLOUD_API_TOKEN=replace_with_permanent_meta_token
WHATSAPP_PHONE_NUMBER_ID=replace_with_phone_number_id
WHATSAPP_TEMPLATE_NAME=gac_login_otp
WHATSAPP_TEMPLATE_LANGUAGE=en

UPSTASH_REDIS_REST_URL=https://replace.upstash.io
UPSTASH_REDIS_REST_TOKEN=replace_with_redis_token

OTP_HASH_SECRET=replace_with_at_least_32_random_bytes
CUSTOMER_SESSION_SECRET=replace_with_another_random_secret
CUSTOMER_SESSION_TTL_SECONDS=604800

APP_ORIGIN=http://localhost:3000
NODE_ENV=development
```

Generate independent secrets in PowerShell

```
$bytes = New-Object byte[] 48
[Security.Cryptography.RandomNumberGenerator]::Create().GetBytes($bytes)
[Convert]::ToBase64String($bytes)
```

Run the command twice. Use one output for OTP_HASH_SECRET and the other for CUSTOMER_SESSION_SECRET.

Graph API version

Keep the Graph API version in configuration instead of hardcoding it. Review Meta's version lifecycle and update the value before the configured version is retired.

STEP 9

Normalize customer phone numbers

The current React form accepts a 10-digit Indian number. The server must normalize it to E.164 and must not trust frontend normalization.

```
import { parsePhoneNumberFromString } from "libphonenumber-js";

export function normalizeIndianPhone(input: string): string {
  const raw = input.trim();
  const phone = parsePhoneNumberFromString(raw, "IN");

  if (!phone?.isValid()) {
    throw new Error("Invalid mobile number");
  }

  return phone.number; // Example: +919876543210
}
```

- Use the normalized value in every Redis key.
- Store phone numbers as text, never as numeric values.
- Remove the plus sign only when the Meta recipient field requires digits only.

STEP 10

Generate and hash the OTP

```
import crypto from "node:crypto";

const otp = crypto.randomInt(100000, 1000000).toString();

const otpHash = crypto
  .createHmac("sha256", process.env.OTP_HASH_SECRET!)
  .update(`${phoneE164}:${otp}`)
  .digest("hex");
```

Store only the HMAC hash. Never log or persist the plaintext code.

```
await redis.set(
  `otp:login:${phoneE164}`,
  {
    hash: otpHash,
    attempts: 0,
    createdAt: Date.now()
  },
  { ex: 300 }
);
```

Required Redis controls

Control	Recommended limit
OTP expiry	5 minutes
Resend cooldown	1 request per phone every 30 seconds
Phone send limit	5 requests per hour
IP send limit	20 requests per hour
Attempts per OTP	5 incorrect attempts
Verify limit	10 requests per IP per 15 minutes
Abuse lock	30 minutes
Control	Recommended limit
OTP expiry	5 minutes
Resend cooldown	1 request per phone every 30 seconds
Phone send limit	5 requests per hour
IP send limit	20 requests per hour
Attempts per OTP	5 incorrect attempts
Verify limit	10 requests per IP per 15 minutes
Abuse lock	30 minutes

STEP 11

Send the authentication template through Meta

Send an HTTPS POST request to the configured Graph endpoint:

```
POST https://graph.facebook.com/v20.0/{PHONE_NUMBER_ID}/messages
Authorization: Bearer {WHATSAPP_CLOUD_API_TOKEN}
Content-Type: application/json
```

A Copy Code authentication template generally uses the following structure. Use the payload example generated for the approved GAC template as the final authority.

```
{
  "messaging_product": "whatsapp",
  "recipient_type": "individual",
  "to": "919876543210",
  "type": "template",
  "template": {
    "name": "gac_login_otp",
    "language": { "code": "en" },
    "components": [
      {
        "type": "body",
        "parameters": [
          { "type": "text", "text": "123456" }
        ]
      },
      {
        "type": "button",
        "sub_type": "url",
        "index": "0",
        "parameters": [
          { "type": "text", "text": "123456" }
        ]
      }
    ]
  }
}
```

A successful response contains a message ID beginning with wamid. If Meta returns a definite non-2xx response, delete the newly stored OTP, log the provider error code server-side and return a generic failure to the browser.

STEP 12

Implement POST /api/auth/otp/request

Request body:

```
{ "phone": "9876543210" }
```

- 1 Reject non-POST methods and oversized or malformed JSON.
- 2 Normalize the phone to E.164.
- 3 Extract the first IP from x-forwarded-for.
- 4 Apply the IP limit, phone limit and 30-second resend cooldown.
- 5 Generate the six-digit code and HMAC hash.
- 6 Store the hash in Redis with a 300-second TTL.
- 7 Send the approved authentication template through Meta.
- 8 Return HTTP 202 with a generic response.

```
{
  "ok": true,
  "message": "Verification code sent.",
  "expiresIn": 300
}
```

Account-enumeration protection

Do not reveal whether the submitted number is registered. A registered-customer database check can be added later while preserving a generic response.

STEP 13

Implement POST /api/auth/otp/verify

```
{
  "phone": "9876543210",
  "code": "123456"
}
```

- 1 Require exactly six digits.
- 2 Normalize the phone and apply verification rate limits.
- 3 Read otp:login:{phone} from Redis.
- 4 Reject missing, expired, blocked or exhausted challenges.
- 5 HMAC the submitted value and compare it using crypto.timingSafeEqual.
- 6 Increment attempts atomically while preserving the remaining TTL after a failure.
- 7 Delete the OTP immediately after a successful comparison.
- 8 Create the customer session and return success.

STEP 14

Issue a temporary secure customer session

```
const sessionToken = crypto.randomBytes(32).toString("base64url");
```

Store only a SHA-256 hash of the token in Redis with a seven-day TTL. The value should contain the verified E.164 phone and creation time.

```
Set-Cookie: gac_customer_session=;  
HttpOnly;  
Secure;  
SameSite=Lax;  
Path=/  
Max-Age=604800
```

- Do not store the session in localStorage.
- Disable Secure only for local HTTP development.
- Invalidate the Redis session during logout.
- Replace this temporary session with the full customer-auth implementation later.

STEP 15

Replace the current dummy React flow

The existing UI uses four OTP inputs. Change it to six:

```
const [otp, setOtp] = useState(["", "", "", "", "", ""]);
```

Send OTP

```
const response = await fetch("/api/auth/otp/request", {  
  method: "POST",  
  headers: { "Content-Type": "application/json" },  
  credentials: "include",  
  body: JSON.stringify({ phone: loginPhone })  
});
```

Verify OTP

```
const response = await fetch("/api/auth/otp/verify", {  
  method: "POST",  
  headers: { "Content-Type": "application/json" },  
  credentials: "include",  
  body: JSON.stringify({  
    phone: loginPhone,  
    code: otp.join("")  
  })  
});
```

Open the dashboard only after the verification endpoint succeeds. Remove every Dummy OTP message and the current any-four-digits behavior.

STEP 16

Configure and deploy on Vercel

Open Vercel -> Project -> Settings -> Environment Variables and add:

```
WHATSAPP_GRAPH_VERSION
WHATSAPP_CLOUD_API_TOKEN
WHATSAPP_PHONE_NUMBER_ID
WHATSAPP_TEMPLATE_NAME
WHATSAPP_TEMPLATE_LANGUAGE
UPSTASH_REDIS_REST_URL
UPSTASH_REDIS_REST_TOKEN
OTP_HASH_SECRET
CUSTOMER_SESSION_SECRET
CUSTOMER_SESSION_TTL_SECONDS
APP_ORIGIN
NODE_ENV
```

- Use test Meta credentials for Preview deployments.
- Use the official sender and permanent token only in Production.
- Do not prefix any secret with REACT_APP_.
- Redeploy after changing environment variables.

Safe rollout order

- 1 Create the Meta app and use the Meta test number.
- 2 Add a controlled test recipient.
- 3 Send a template manually through the official API collection.
- 4 Create Upstash Redis and verify TTL behavior.
- 5 Implement and test otp/request.
- 6 Implement and test otp/verify, reuse rejection and expiry.
- 7 Add the secure session cookie.
- 8 Connect the React login in a Vercel Preview deployment.
- 9 Connect the official business number and approved GAC template.
- 10 Add production credentials and release.

Production acceptance checklist

- The OTP contains exactly six digits.
- The OTP expires after five minutes.
- A successful OTP cannot be reused.
- Resending is blocked for 30 seconds.
- Phone and IP hourly limits are enforced.
- Five incorrect attempts invalidate the OTP.
- The Meta token does not appear in the browser bundle.
- The Redis token does not appear in the browser bundle.
- OTP values never appear in application logs.
- The session cookie is HttpOnly and Secure in production.
- API responses do not reveal whether a phone is registered.
- Login succeeds from two separate devices.
- Meta test credentials are absent from production.
- The Graph API version is configuration-driven.

Common troubleshooting

Meta returns template not found

Confirm the exact approved template name, language code and WhatsApp Business Account. Template names are case-sensitive and the sender must belong to the same account.

Meta accepts the request but no WhatsApp arrives

Check that the recipient is a registered test recipient during development, inspect the returned wamid, confirm the number uses country code digits and review the message status in WhatsApp Manager.

Authentication template is unavailable or rejected

Verify business eligibility, payment configuration and template category. Use Meta's preset Authentication category with a supported OTP button rather than a custom utility message.

OTP always appears expired

Confirm the application is using the same Upstash database and normalized E.164 key in both endpoints. Check the key TTL immediately after request and verify that Preview and Production variables are not mixed.

Works locally but fails on Vercel

Check Vercel function logs, environment scope, APP_ORIGIN, Secure-cookie behavior and whether every secret was followed by a redeploy.

Official references

Use these sources to confirm current dashboard labels, API payloads and platform behavior before production rollout.

- Meta WhatsApp Cloud API official Postman collection: postman.com/meta/whatsapp-business-platform
- Meta for Developers: developers.facebook.com
- Meta Business Manager: business.facebook.com
- Upstash Redis connection guide: upstash.com/docs/redis/howto/connect-with-upstash-redis
- Upstash TypeScript SDK: upstash.com/docs/redis/sdks/ts/getstarted
- Upstash rate-limit SDK: upstash.com/docs/redis/sdks/ratelimit-ts/overview
- Vercel environment variables: vercel.com/docs/projects/environment-variables

Next implementation action

After the Meta test sender, approved template, Phone Number ID and Upstash database are ready, implement the two Vercel endpoints and replace the current four-digit dummy flow with the six-digit API-backed flow.